

TypeScript Part=2

1.any

- Means: anything is allowed
- TypeScript does no checking
- Can cause errors later

Example:

```
let person: any;
```

```
person = "Vijay";
```

```
person = 25;
```

```
console.log(person.toUpperCase()); // runtime crash if person = 25
```

any allows everything, even wrong things.

2.unknown

- Means: value is unknown
- You must check the type first
- Much safer

Example:

```
let person: unknown;
```

```
person = "Sanya";
```

```
person = 25;
```

```
// person.toUpperCase(); not allowed
```

```
if (typeof person === "string") {
```

```
    console.log(person.toUpperCase()); // safe
```

```
}
```

unknown says: check first, then use

3.Difference (one line)

- any: use directly, no safety
- unknown: check first, safe

4.Compiler

A compiler converts all code at once into machine code before running.

How it works

Source Code → Compiler → Machine Code → Run

Example

Languages:

- C
- C++
- TypeScript (tsc)

```
// one.ts
```

```
let a: number = 10;
```

```
console.log(a);
```

Run:

```
tsc one.ts // compile first
```

```
node one.js // then run
```

Pros

- Fast execution
- Errors found before run

Cons

- Need to compile first
- Takes time for big programs

5.What is an Interpreter?

An interpreter runs code line by line

No separate compile step

How it works

Source Code → Run line by line

Example

Languages:

- JavaScript
- Python
- Ruby

let x = 10;

console.log(x);

Run directly:

node app.js

Pros

- Easy to run
- Good for beginners

Cons

- Slower than compiled code
- Error comes during execution

6.What is JIT?

JIT is a mix of compiler + interpreter

Code is:

- Interpreted first
- Then frequently used parts are compiled at runtime

How it works

Code → Interpreter → Hot code → JIT Compiler → Fast Machine Code

Example

Languages using JIT:

- Java (JVM)
- JavaScript (V8 engine)
- C# (.NET)

JavaScript example

```
for (let i = 0; i < 1000000; i++) {
```

```
// repeated code
```

```
}
```

JIT says:

“This loop runs many times, compile it for speed”

Pros

- Faster than interpreter
- No full compile wait
- Best performance

Cons

- More memory usage
- Complex system

Feature	Compiler	Interpreter	JIT
Convert code	All at once	Line by line	During runtime
Speed	Fast	Slow	Very fast
Errors	Before run	During run	Both
Example	C, TS	JS, Python	Java, JS

- **Platform-dependent language:**
A language whose program runs only on a specific operating system or hardware.
- **Platform-independent language:**
A language whose program runs on any operating system or hardware without change.

Example

- Platform-dependent: C, C++
- Platform-independent: Java, Python, JavaScript

7.What is a Transpiler?

- Transpiler: A tool that converts code from one programming language to another similar-level language.

Example

- TypeScript → JavaScript
- ES6 JavaScript → ES5 JavaScript

A transpiler does not convert to machine code, only source code to source code.

8.Type vs Interface (Main Differences)

Point	type	interface
Use	Any type (primitive, union, object)	Mainly objects & classes
Primitive	Yes	No
Union ()	Yes	No
Intersection (&)	Yes	No
Declaration merging	No	Yes
Extend / Inherit	& (intersection)	extends
Classes support	Limited	Best
Readability	Good	Better for large projects

9.What is a Tuple? (TypeScript)

A tuple is a special type of array in TypeScript where the number of elements and their types are fixed and ordered.

Example

```
let user: [string, number] = ["Vijay", 23];
```

First value must be string

Second value must be number

Order matters

Invalid examples

```
let user: [string, number] = [23, "Vijay"]; // wrong order
```

```
let user: [string, number] = ["Vijay"]; // missing value
```

A tuple allows storing multiple values of different types in a fixed order.